

1 Le langage Python

1.1 Règles typographiques

Variables

- Des noms explicites
- mots en lettres minuscules séparés par des `_` pour les variables et les fonctions
- Possibilité d'utiliser des majuscules pour des constantes.

Code

- indentation de 4 espaces (touche tabulation par défaut)
- lignes vides avant et après chaque fonction
- espaces autour du `=` pour les variables (mais pas pour les arguments par défaut d'une fonction)
- éviter les lignes trop longues : soit avec des variables intermédiaires, soit en distribuant la fonction sur plusieurs lignes (le plus simple est de le faire avec des parenthèses).

1.2 Types de données

Les nombres

Il existe 3 types de nombres en Python : entiers (taille infinie), les nombres à virgule flottante, les complexes

- Calculer `3**523` ?
- Quel est la quotient et le reste de la division euclidienne de 2026 par 157 ?
- Entrer sous forme d'un littéral le nombre suivant écrit en hexadécimal : `3B1` ? (au fait, qu'est-ce qu'un littéral ?)

En Python, les nombres à virgule flottante sont stockés sur 64 bits : 1 bit de signe, 11 bits pour l'exposant et 52 pour la mantisse.

- Entrer la constante de Planck sous forme d'un littéral $6,62607015 \times 10^{-34}$
- Quels sont en ordre de grandeur le plus petit et le plus grand nombre (en valeur absolu) que l'on peut représenter ?
- Que vaut $(1 + 1E-16) - 1$? Que vaut $(1 + 1E-15) - 1$? Comparez à 5×2^{-52} ? Pourquoi ?

Les nombres complexes sont enregistrés sous la forme de deux nombres réels.

- Créer le nombre $z = 1 + 3i$?
- Que vaut la partie réelle de z^3 ?
- Comment calculer la z^* le conjugué complexe de z ?

Note : lorsque l'on utilise la syntaxe `z.imag`, on dit que `imag` est un attribut de `z`. Lorsque l'on utilise la syntaxe `z.conjugate()`, on dit que `conjugate` est une méthode.

Booléens

- Que vaut `True or False and False` ? (`True or False`) `and False` ou `True or (False and False)` ?
- Pourquoi seule la seconde ligne du code suivant produit une erreur ?

```
from math import sqrt
x = -1

if x>=0 and sqrt(x)>2:
    print('x est plus grand que sqrt(2)')

if (x>=0) & (sqrt(x)>2):
    print('x est plus grand que sqrt(2)')
```

- Pourquoi `15>16 & 8>3` est faux mais `15>15 & 8>3` est vrai ?

```
print(15>16 & 8>3)
print(15>15 & 8>3)
```

1.3 Chaînes de caractères (str)

Exemples

Les chaînes de caractères peuvent être créées à l'aide de `"` ou `'` ou `"""` ou `'''`.

- Créer simplement une chaîne contenant le mot `Aujourd'hui`
- Comment créer le littéral suivant : Voici le mot `"Aujourd'hui"` entre guillemets !

Les caractères spéciaux tels que `\n` (retour à la ligne) ou `\t` (tabulation) correspondent à des caractères qui ne sont pas affichés (à l'origine, ce sont des "touches" sur une machine à écrire).

Caractères spéciaux

- En utilisant le `\n`, écrire une chaîne sur deux lignes. Quelle est la longueur de la chaîne ?
- En utilisant une raw string, créer la chaîne suivante (utilisée en LaTeX) `\[ k=2\pi/\lambda \]`

Unicode

- Créer en utilisant l'Unicode la chaîne suivante : La constante de Planck réduite, notée $\hbar$, est une constante universelle, non relative, qui a la même dimension qu'une énergie multipliée par une durée.
- Afficher l'ensemble des 25 lettres de l'alphabet grec (on utilisera la fonction `chr`, et le fait que le α correspond à l'Unicode 03B1).

Il est possible en Python d'utiliser l'unicode dans des noms de variable. Ceci est très fortement déconseillé.

- Expliquer pourquoi le code suivant n'affiche pas ce à quoi on pense ?

```
α = 1/137.03
A = 2
```

(continues on next page)

```
A = 1
print(A)
```

Méthodes

Nous allons utiliser la chaîne de caractère ci-dessous :

```
s = "La théorie de la relativité restreinte et générale"
```

- Mettre cette chaîne en majuscules
- Où se trouve le premier `r` de la chaîne ?
- Remplacer les `a` par des `z`

On considère la liste et la chaîne suivantes. Convertir la liste en chaîne et réciproquement (on utilisera des list comprehension)

```
s = "1;2;3;4"
l = [4, 5, 6]
```

Formatage

On considère les variables `pi` et `c` suivantes :

```
from math import pi
c = 299792458
```

- Afficher en utilisant le formatage de chaîne de caractères les chaînes suivantes La valeur de `pi` est approximativement 3.1416 et La vitesse de la lumière est `c=2.998e+08 m/s`.
- Compléter la fonction suivante afin d'afficher :

```
21h25
3h05
```

```
def afficher_heure(heure, minute):
    print("...")
```

```
afficher_heure(21, 25)
afficher_heure(3, 5)
```

1.4 Listes

Création et modification

- On reprend l'exemple vu la semaine dernière. Pourquoi le comportement est différent entre une liste et un tableau numpy ? Est-ce que tous les éléments d'une liste doivent être du même type ?

```
a = [1, 2, 4]
a[1] = 3.14
print(a)
```


Les dictionnaires

- Voici un dictionnaire qui contient les paramètres d'un problème. Afficher le paramètre l . Parcourir le dictionnaire pour afficher tous les paramètres
- Écrire une fonction qui prend ces paramètres et renvoie la période du pendule ($2\pi\sqrt{l/g}$)

```
parametres = {"N":100, 'l':30, "h":50, "g":9.81, 'm':0.1}
```

- Quels sont les types de données qui peuvent être utilisées comme clé d'un dictionnaire ?

1.7 Fonctions

On considère la fonction suivante :

```
def f(x, a=1, b=0):  
    return a*x + b
```

- Calculer f en $x = 1$ pour $a = 3$ et $b = 1$ en utilisant 1/ des arguments ordonnés, 2/ des arguments par clé 3/ des arguments a et b dans un dictionnaire.
- modifier la fonction qui calcule la période du pendule pour qu'on puisse l'utiliser soit avec un dictionnaire soit simplement avec les arguments l et g (lequel vaut 9.81 par défaut):

```
periode(**parametres)  
periode(3)  
periode(l=3, g=10, m=4)
```

- Écrire une fonction `somme` qui prend un nombre arbitraire d'arguments et en renvoie la somme (faire la boucle explicitement)
- Écrire une fonction `primitive` qui prend comme argument une fonction et renvoie une fonction qui est une primitive de la fonction calculée numériquement. Le nombre de pas sera une argument optionnel. On prendra un algorithme quelconque (ce n'est pas le problème). On doit pouvoir faire :

```
def f(x):  
    return 1/(1+x**2)  
F = primitive(f)  
print(F(1))  
  
F = primitive(f, N=10000)  
print(F(1))
```

1.8 Fichiers

Fichiers en python

- Pourquoi faut-il utiliser un context manager (`with open(...) as ...:`) pour ouvrir un fichier
- A l'aide de Python, créer un fichier contenant le mot "Bonjour".
- Ouvrir ce fichier à l'aide d'un éditeur de texte et le modifier.
- Lire à nouveau le fichier avec Python

Fichiers avec numpy

- Créer un tableau numpy contenant 10 000 000 de points. Enregistrer le fichier avec la commande `np.save` et `np.savetxt`. Comment lire le fichier ? Quelle est la méthode la plus rapide ? Pourquoi ? Quel est la taille du fichier créé par `np.save`.
- Avec un tableur type openoffice/excel, créer un fichier contenant des données sur deux colonnes. Enregistrer le fichier en `.csv`. Lire le fichier avec Python

Format JSON

- Voici un annuaire. Enregistrer le sous forme de fichier `json` grâce au module `json` de Python.

```
# Liste de dictionnaires
personne_1 = {"nom": "Dupont", "prenom": "Archibald", "age": 13}
personne_2 = {"nom": "Dubois", "prenom": "Joséphine", "age": 34}

annuaire = [personne_1, personne_2]
```

2 Exercice

2.1 Liste de course

Voici un panier de courses sous forme d'une liste de tuples.

```
panier = [
    ('tomates', 2, 4.5),
    ('courgettes', 1, 3.70),
    ('poivron', 0.5, 8.5)]
```

Afficher la liste sous la forme suivante :

Tomates		2.00 kg		4.50€/kg		9.00€	
Courgettes		1.00 kg		3.70€/kg		3.70€	
Poivron		0.50 kg		8.50€/kg		4.25€	

3 Analyse des données du CAC 40

L'objectif de cet exercice est de manipuler des données à l'aide de listes et de dictionnaires (sans utiliser de tableaux numpy).

Nous avons téléchargé depuis le site <https://fr.finance.yahoo.com/quote/%5EFCHI/history/> les données du CAC 40 depuis 3 ans (fichier `data/cac40.csv`). Le fichier obtenu contient : la date, le cours d'ouverture, le plus haut, le plus bas, le cours de clôture, le cours de clôture ajusté et le volume de titres échangés. Ce contenu est séparé par des `,`.

Remarque : la méthode `split` d'une chaîne de caractère permet de la séparer en plusieurs sous chaînes. Par exemple :

```
s = '12,bonjour,3.14'
print(s.split(','))
```

On supprimera la première ligne du fichier.

1. Afficher les 10 premières lignes du fichier
2. Créer une liste contenant le plus haut de la bourse jour par jour. Qu'elle est la valeur la plus haute du CAC 40 ?
3. Écrire une fonction qui prend le jour, le mois et l'année et renvoie une chaîne de caractère pour la date sous la forme : `aaaa-mm-jj` (par exemple `1515-09-13`)
4. Écrire une fonction qui renvoie le plus bas d'une journée à partir du jour, du mois et de l'année. Cette fonction renverra une exception si le jour n'existe pas.
5. Plutôt qu'une liste, on va utiliser un dictionnaire dont la clé sera la date. Créer un dictionnaire qui contiendra pour chaque jour un dictionnaire avec pour clé : `'haut'`, `'bas'`, `'ouverture'`, `'fermeture'`, `'volume'`. Par exemple on pourra utiliser : `cac_40['2020-10-05']['haut']`
6. Enregistrer ce dictionnaire dans un fichier au format json. Vérifier que vous pouvez l'ouvrir !
7. Regarder la documentation de la fonction `parse` du module `dateutil.parser` et trouver le moyen de convertir une date sous la forme `2020-10-05` en un nombre (nombre de jour ou nombre de seconde depuis une date donnée).
8. Tracer l'évolution au cours du temps de CAC 40.